

Table of contents

Série 10 : Interprétation Abstraite II/III	1
Méthodes	1
Analyse des structures d'ordre	1
Analyse des fonctions sur des treillis	1
Connexions de Galois (Galois Connections)	2
Interprétation abstraite de programmes	2
Rappel	2
Exemples Illustratifs	2
Exemple 1 : Vérification qu'un poset est un treillis complet	2
Exemple 2 : Définition d'un ordre et calcul d'inf/sup sur le domaine des intervalles	3
Exemple 3 : Preuve qu'une fonction Scott-continue est monotone	3
Exemple 4 : Construction d'une Connexion de Galois entre $\mathcal{P}(\mathbb{Z})$ et $\mathcal{I}$	4
Exemple 5 : Analyse de programme avec le treillis des intervalles	4
Synthèse	5

Série 10 : Interprétation Abstraite II/III

Méthodes

Analyse des structures d'ordre

- **Vérification des propriétés de treillis** : Déterminer si un poset donné est un treillis, un treillis complet, ou un treillis semi-complet en vérifiant l'existence d'infimum et de supremum pour toute paire (treillis) et pour tout sous-ensemble (complet).
- **Construction d'ordre sur des domaines abstraits** : Définir un ordre qui capture la notion de « précision » entre éléments abstraits (ex. : inclusion pour les intervalles).
- **Produit cartésien de treillis** : Montrer que le produit de deux treillis complets, muni de l'ordre produit, est lui-même un treillis complet.

Analyse des fonctions sur des treillis

- **Monotonie et continuité de Scott** : Prouver qu'une fonction est monotone et/ou Scott-continue. La continuité de Scott garantit la préservation des suprémums de chaînes croissantes.
- **Composition de fonctions continues** : Montrer que la composée de deux fonctions Scott-continues est Scott-continue.
- **Calcul de points fixes** : Appliquer le théorème de Kleene pour calculer le point fixe le plus bas (lfp) d'une fonction continue, souvent via l'itération à partir de $\perp$.

- **Comparaison de points fixes** : Si $f(x) \leq g(x)$ pour tout x , alors $\text{lfp}(f) \leq \text{lfp}(g)$.

Connexions de Galois (Galois Connections)

- **Définition des fonctions α (abstraction) et γ (concrétisation)** entre un domaine concret et un domaine abstrait.
- **Preuve des propriétés fondamentales** : Vérifier que (α, γ) forment bien une connexion de Galois, i.e., $\alpha(X) \subseteq i \Leftrightarrow X \subseteq \gamma(i)$.
- **Propriétés dérivées** : Montrer que α et γ sont monotones, que $x \leq \gamma(\alpha(x))$, et que $\alpha(\gamma(y)) \leq y$.

Interprétation abstraite de programmes

- **Construction du système d'équations** : À partir d'un CFG (Control Flow Graph), écrire les équations de point fixe pour chaque point de programme.
- **Résolution itérative** : Appliquer l'itération de Kleene sur un domaine abstrait (ex. : treillis des signes, des intervalles) jusqu'à convergence.
- **Traitement des conditions** : Définir l'opérateur de "filtrage" ($\cap[\text{condition}]$) pour raffiner l'information à une garde (ex. : $x \geq y$ dans le domaine des intervalles).

Rappel

- **Treillis complet** : Tout sous-ensemble a un infimum et un supremum.
- **Continuité de Scott** : $f(\sup_i u_i) = \sup_i f(u_i)$ pour toute chaîne croissante (u_i) .
- **Théorème de Kleene** : Dans un treillis complet, une fonction Scott-continue a un plus petit point fixe donné par $\sup_{n \geq 0} f^n(\perp)$.
- **Connexion de Galois** : Paire $(\alpha : \mathcal{C} \rightarrow \mathcal{A}, \gamma : \mathcal{A} \rightarrow \mathcal{C})$ vérifiant $\forall X \in \mathcal{C}, \forall i \in \mathcal{A}, \alpha(X) \subseteq i \Leftrightarrow X \subseteq \gamma(i)$.
- **Ordre produit** : $(a, b) \leq (c, d) \Leftrightarrow a \leq c \wedge b \leq d$. Infimum et supremum se calculent composante par composante.

Exemples Illustratifs

Exemple 1 : Vérification qu'un poset est un treillis complet

Contexte : L'exercice demande si $(\mathbb{N}, \leq)$ est un treillis (complet).

Application :

- C'est un treillis car pour toute paire $\{a, b\}$, $\inf(a, b) = \min(a, b)$ et $\sup(a, b) = \max(a, b)$ existent dans $\mathbb{N}$.

- Il n'est **pas complet** car le sous-ensemble $X = \mathbb{N}$ n'a pas de supremum dans $\mathbb{N}$ ($\sup(\mathbb{N}) = +\infty \notin \mathbb{N}$).
- C'est un **treillis semi-complet** car tout sous-ensemble $A \subseteq \mathbb{N}$ a un infimum (son minimum).
- En revanche, $(\mathbb{N} \cup \{\infty\}, \leq)$ est un treillis complet.

Exemple 2 : Définition d'un ordre et calcul d'inf/sup sur le domaine des intervalles

Contexte : Pour l'ensemble d'intervalles $\mathcal{I}$, définir un ordre représentant la perte de précision.

Application :

- On définit l'ordre par la **contre-inclusion** des ensembles de valeurs : $(a, c) \leq (b, d) \Leftrightarrow [b, d] \supseteq [a, c] \Leftrightarrow (b \leq a \wedge c \leq d)$.
- L'infimum correspond à l'**intersection** des ensembles de valeurs :

$$\inf((a, c), (b, d)) = (\max(a, b), \min(c, d))$$

(ce qui donne l'intervalle le plus petit contenu dans les deux).

- Le supremum correspond à l'**union** (la plus petite sur-approximation) :

$$\sup((a, c), (b, d)) = (\min(a, b), \max(c, d))$$

- On prouve que $(\mathcal{I}, \leq)$ est un treillis complet en utilisant cette correspondance avec l'inclusion d'ensembles.

Exemple 3 : Preuve qu'une fonction Scott-continue est monotone

Contexte : Soit $f : E \rightarrow F$ Scott-continue, avec E, F des treillis complets.

Application : Soit $x, y \in E$ tels que $x \leq y$.

1. Construire la chaîne croissante (u_i) : $u_0 = x$, et $u_i = y$ pour tout $i \geq 1$.
2. Par construction, $\sup_i u_i = y$.
3. Par continuité de Scott : $f(y) = f(\sup_i u_i) = \sup_i f(u_i)$.
4. En particulier, $f(u_0) = f(x) \leq \sup_i f(u_i) = f(y)$.
5. Donc $f(x) \leq f(y)$, ce qui prouve la monotonie.

Exemple 4 : Construction d'une Connexion de Galois entre $\mathcal{P}(\mathbb{Z})$ et $\mathcal{I}$

Contexte : Définir α et γ entre l'ensemble des parties de $\mathbb{Z}$ (concret) et le treillis des intervalles $\mathcal{I}$ (abstrait).

Application :

- **Fonction d'abstraction α :**

$$\alpha : \mathcal{P}(\mathbb{Z}) \rightarrow \mathcal{I}, \quad \alpha(X) = [\inf X, \sup X]$$

(avec $\inf \emptyset = +\infty$, $\sup \emptyset = -\infty$ par convention).

- **Fonction de concrétisation γ :**

$$\gamma : \mathcal{I} \rightarrow \mathcal{P}(\mathbb{Z}), \quad \gamma([a, b]) = \{z \in \mathbb{Z} \mid a \leq z \leq b\}$$

- **Preuve de la connexion :** Il faut montrer $\forall X \subseteq \mathbb{Z}, \forall i \in \mathcal{I} :$

$$\alpha(X) \subseteq i \Leftrightarrow X \subseteq \gamma(i)$$

1. ($\Rightarrow$) Si $[\inf X, \sup X] \subseteq i$, alors tout $x \in X$ vérifie $x \in \gamma(i)$.
2. ($\Leftarrow$) Si $X \subseteq \gamma(i)$, alors $\inf X \geq a$ et $\sup X \leq b$, donc $[\inf X, \sup X] \subseteq [a, b] = i$.

Exemple 5 : Analyse de programme avec le treillis des intervalles

Contexte : Analyser le programme suivant avec le domaine des intervalles :

```
y := 3; x := 4;
while (x >= y) do x := x - 4; done;
```

Application :

1. CFG et équations :

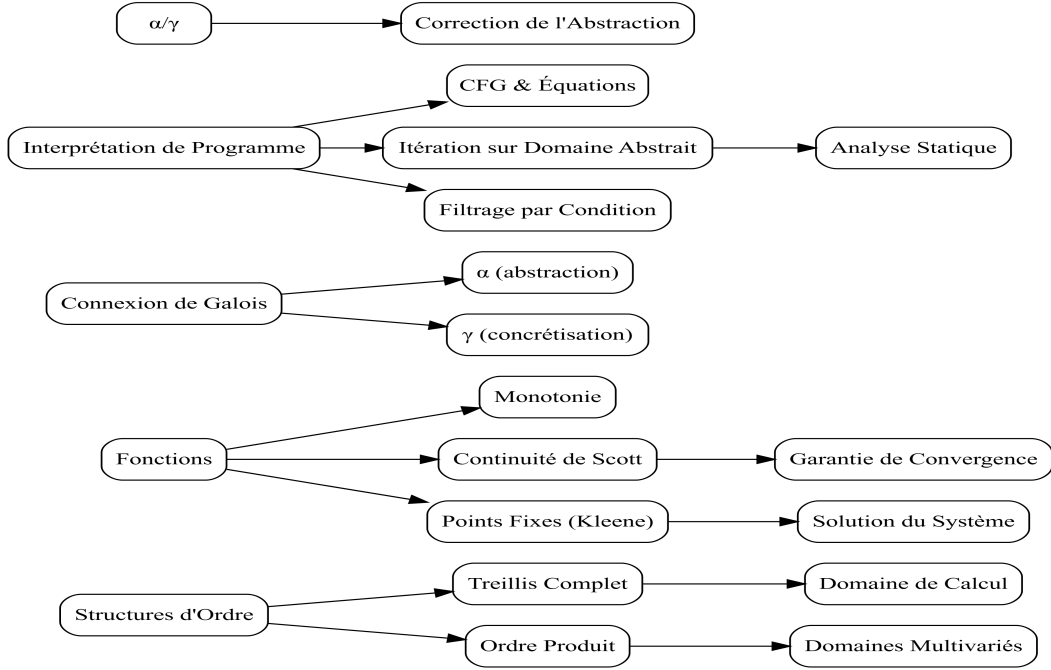
- Entrée : $v_0 = (x : \top, y : \top)$
- $v_1 = (x : [4, 4], y : [3, 3])$ après les affectations.
- Boucle (point v_2) : $v_2 = v_1 \sqcup F(v_2)$, où F est la fonction de transfert du corps.
- Corps de boucle : $F((x, y)) = (x - 4, y) \cap [x \geq y]$.

2. Itération de Kleene :

- $v_2^{(0)} = \perp = (x : \perp, y : \perp)$
- $v_2^{(1)} = v_1 \sqcup F(\perp) = (x : [4, 4], y : [3, 3])$
- Calcul de $F(v_2^{(1)})$: $(x : [4, 4] - 4, y : [3, 3]) = (x : [0, 0], y : [3, 3])$. La condition $x \geq y$ devient $[0, 0] \geq [3, 3]$ qui est faux, donc le résultat est $\perp$.

- $v_2^{(2)} = v_1 \sqcup \perp = v_2^{(1)}$. Point fixe atteint.

3. **Résultat** : À la sortie de la boucle, $x = [4, 4]$ et $y = [3, 3]$. La boucle ne s'exécute pas.



Synthèse

Cette série met en lumière les fondements mathématiques de l'interprétation abstraite : les **treillis complets** servent de domaine de calcul, la **continuité de Scott** assure la convergence des itérations, et les **connexions de Galois** garantissent la correction de l'abstraction.

La méthode centrale combine la modélisation d'un programme par un **système d'équations de point fixe** et sa résolution itérative sur un domaine abstrait (signes, intervalles).

Ces techniques permettent d'inférer automatiquement des propriétés sur les programmes (bornes de variables, signes) sans les exécuter.